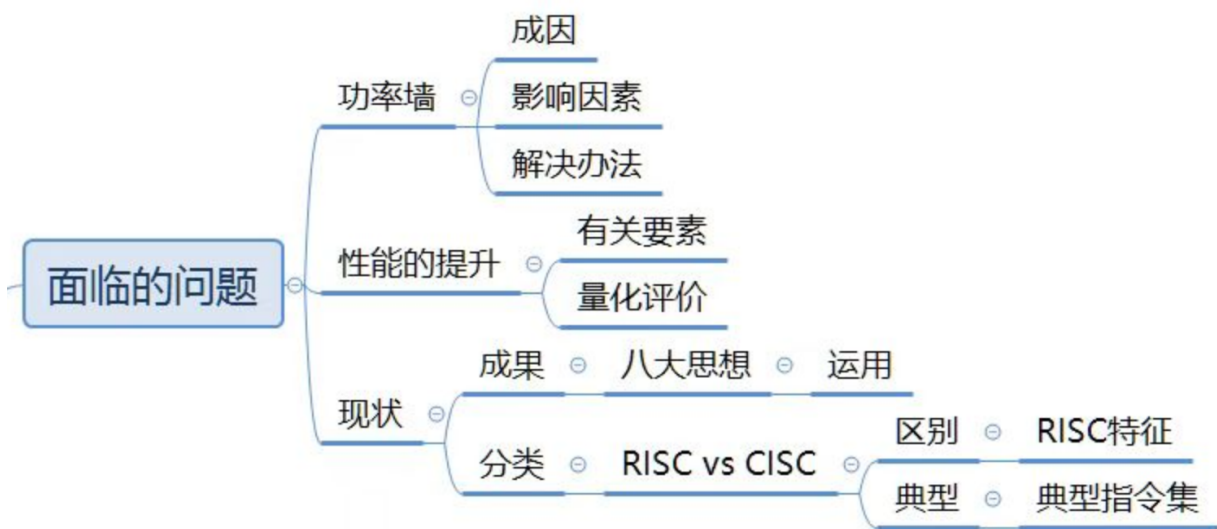
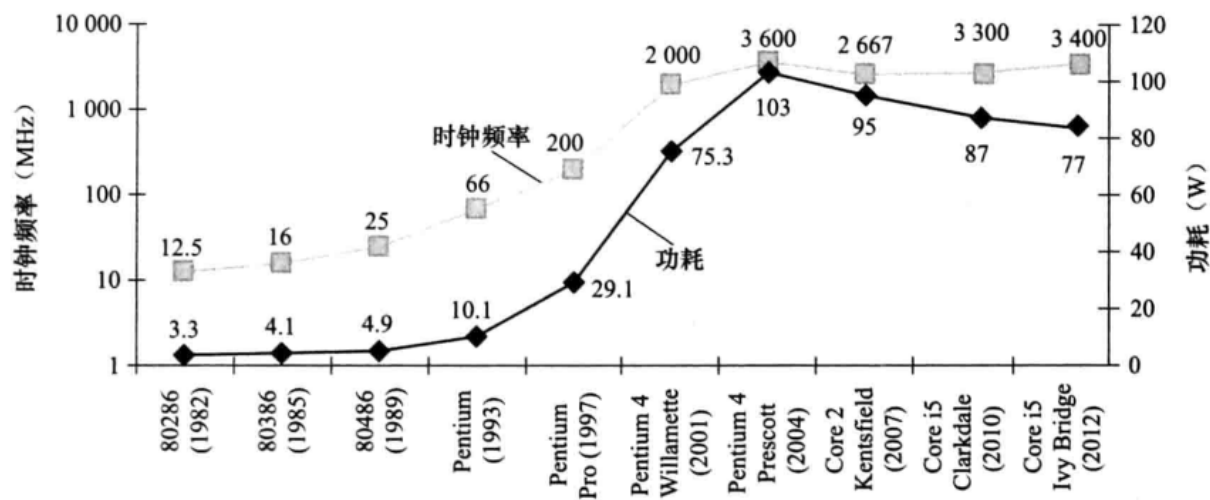


计算机组成原理

计算机发展面临的问题



功率墙



上图是30年间Intel八代微处理器的时钟频率和功耗的增长趋势。两者的增长几乎保持了几十年，但近几年来突然缓和下来。其原因在于两者是密切相关的，而且功耗已经到达了极限，无法再将处理器冷却下来。

性能

- 响应时间：也叫执行时间（execution time），是计算机完成某任务所需的总时间，包括硬盘访问、内存访问、I/O活动、操作系统开销和CPU执行时间等。
- 吞吐率：也叫带宽（bandwidth），性能的另一度度量参数，表示单位时间内完成的任务数量。

CPU执行时间（CPU execution time），简称CPU时间：只表示在CPU上花费的时间，而不包括等待I/O或运行其他程序的时间。（需要注意的是，用户所感受到的是程序的响应时间，而不是CPU时间。）CPU时间还可进一步分为用于用户程序的时间和操作系统为用户服务花去的CPU时间。前者称为用户CPU时间（user CPU time），后者称为系统CPU时间（system CPU time）。

- CPU执行时间：简称CPU时间，执行某一任务在CPU上所花费的时间。
- 用户CPU时间：在程序本身所花费的CPU时间。
- 系统CPU时间：为执行程序而花费在操作系统上的时间。
- 系统性能：表示空载系统的响应时间；
- CPU性能：用户CPU时间

一个程序的CPU执行时间 = 一个程序的CPU时钟周期数 × 时钟周期时间

一个程序的CPU执行时间 = 一个程序的CPU时钟周期数 / 时钟频率

CPI：每条指令的时钟周期数，表示执行某个程序或者程序片段时每条指令所需的时钟周期平均数。

CPU时间 = 指令数 × CPI × 时钟周期时间

CPU时间 = 指令数 × CPI / 时钟频率

性能的分量	测量单位
程序的CPU执行时间	程序执行的执行时间，以秒为单位
指令数目	程序执行的指令数目
指令的平均执行时钟周期(CPI)	每条指令的平均执行的时钟周期数
时钟周期时间	每个时钟周期的长度，以秒为单位

只用一种因素（如指令数）去评价性能是危险的。当比较两台计算机时，必须考虑全部三个因素，它们组合起来才能确定执行时间。如果某个因素相同（如时钟频率），则必须考虑不同的因素才能确定性能的优劣。因为CPI随着指令组合（instruction mix）而变化，所以必须比较指令的条数和CPU，即使时钟频率是相同的。

Amdahl定律：

改进后的执行时间=受改进影响的执行时间/改进量 + 不受影响的执行时间

Amdahl定律：阐述了“对于特定改进的性能提升可能由所使用的改进特征的数量所限制”的规则。它是“收益递减定律”的量化版本。

现状

一、八大思想：

- **面向摩尔定律的设计**

摩尔定律（Moore's Law）指出单芯片上的集成度每18~24个月翻一番。由于计算机设计需要几年时间，因此在项目结束时，单芯片的集成度相对于设计开始时很容易翻一番甚至翻两番。

- **使用抽象简化设计**

使用抽象来表达不同的设计层次，在高层次中看不到低层次的细节，只能看到一个简化的模型。

- **加速大概率事件**

加速大概率事件远比优化小概率事件更能提高性能。大概率事件通常比小概率事件简单，易于提高。这就要求设计者需要知道什么事件是经常发生的。

- **通过并行提高性能**

- **通过流水线提高性能**

- **通过预测提高性能**

假定从误预测恢复代价不高，并且预测的准确率相对较高，则通过猜测的方式提前开始某些操作，要比等到确切知道这些操作应该启动时才开始要快一些。

- **存储器层次**

速度越快的往往容量越小，也越昂贵；速度越慢的往往容量越大，也越便宜。

- **通过冗余提高可靠性**

任何一个物理器件都有可能失效，因此可以通过使用冗余部件的方式提高系统的可靠性。

二、RISC和CISC

RISC：精简指令集计算机。

CISC：复杂指令集计算机

指令系统朝两个截然不同的方向的发展：

- 一是增强原有指令的功能，设置更为复杂的新指令实现软件功能的硬化，这类机器称为复杂指令系统计算机（CISC）
典型的有采用x86架构计算机；
- 二是减少指令种类和简化指令功能，提高指令的执行速度，这类机器称为精简指令统计计算机（RISC）
典型的有ARM、MIPS架构的计算机。

CISC：

随着VLSI技术的发展，硬件成本不断下降，软件成本不断上升，促使人们在指令系统中增加更多、更复杂的指令，以适应不同的应用领域，这样就构成了复杂指令系统计算机(CISC)。

CISC的主要特点如下：

- 1) 指令系统复杂庞大，指令数目一般为200条以上。
- 2) 指令的长度不固定，指令格式多，寻址方式多。
- 3) 可以访存的指令不受限制。

- 4) 各种指令使用频度相差很大。
- 5) 各种指令执行时间相差很大，大多数指令需多个时钟周期才能完成。
- 6) 控制器大多数采用微程序控制。有些指令非常复杂，以至于无法采用硬连线控制。
- 7) 难以用优化编译生成高效的目标代码程序。

RISC:

CISC如此庞大的指令系统，对指令的设计提出了极高的要求，研制周期变得很长。后来人们发现，一味地追求指令系统的复杂和完备程度不是提高计算机性能的唯一途径。对传统CISC指令系统的测试表明，各种指令的使用频率相差悬殊，大概只有20%的比较简单指令被反复使用，约占整个程序的80%；而80%左右的指令则很少使用，约占整个程序的20%。从这一事实出发，人们开始了对指令系统合理性的研究，于是RISC随之诞生。

精简指令系统计算机（RISC）的中心思想是要求指令系统简化，尽量使用寄存器-寄存器操作指令，指令格式力求一致。RISC的主要特点如下：

- 1) 选取使用频率最高的一些简单指令，复杂指令的功能由简单指令的组合来实现。
- 2) 指令长度固定，指令格式种类少，寻址方式种类少。
- 3) 只有Load Store（取数/有数）指令访存，其余指令的操作都在寄存器之间进行。
- 4) CPU中通用寄存器的数量相当多。
- 5) RISC定采用指令流水线技术，大部分指令在一个时钟周期内完成。
- 6) 以硬布线控制为主，不用或少用微程序控制。
- 7) 特别重视编译优化工作，以减少程序执行时间。

从指令系统兼容性看，CISC大多能实现软件兼容，即高档机包含了低档机的全部指令，并可加以扩充。但RISC简化了指令系统，指令条数少，格式也不同于老机器，因此大多数RISC机不能与老机器兼容。

比较:

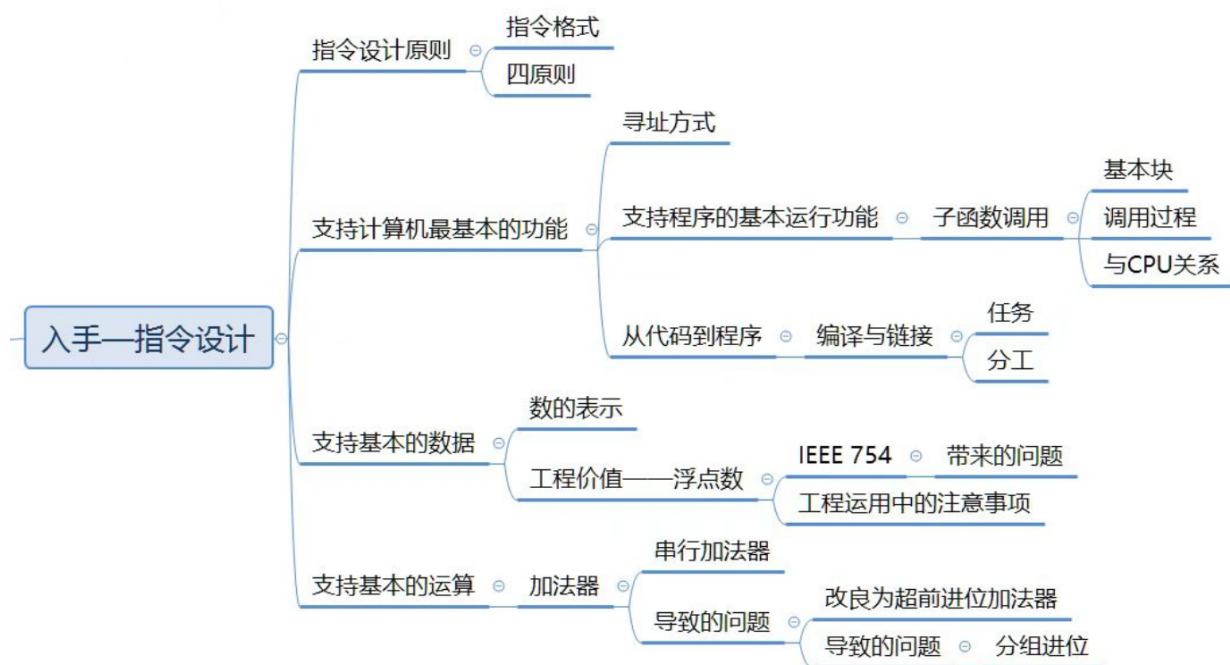
RISC的优点:

- 1) RISC更能充分利用VLSI芯片的面积。CISC的控制器大多采用微程序控制，其控制存储器在CPU芯片内所占的面积达50%以上，而RISC控制器采用组合逻辑控制，其硬布线逻辑只占CPU芯片面积的10%左右。
- 2) RISC更能提高运算速度。RISC的指令数、寻址方式和指令格式种类少，又设有多个通用寄存器，采用流水线技术，所以运算速度更快，大多数指令在一个时钟周期内完成。
- 3) RISC便于设计，可降低成本，提高可靠性。RISC指令系统简单，因此机器设计周期短；其逻辑简单，因此可靠性高。
- 4) RISC有利于编译程序代码优化。RISC指令类型少，寻址方式少，使编译程序容易选择更有效的指令和寻址方式，并适当地调整指令顺序，使得代码执行更高效化。

发展趋势

由于RISC具有更强的实用性，因此应该是未来处理器的发展方向。但事实上，当今时代Intel几乎一统江湖，且早期很多软件都是根据CISC设计的，单纯的RISC将无法兼容。此外，现代CISC结构的CPU已经融合了很多RISC的成分，其性能差距已经越来越小。CISC可以提供更多的功能，这是程序设计所需要的。

入手——指令设计



指令设计原则

一、指令设计四大原则

1. 简单源于规整
2. 越小越快
3. 加速执行常用操作
4. 优秀的设计需要好的折中

二、指令格式

• R型

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

op: 操作码(opcode)

rs: 第一源操寄存器号

rt: 第二源操寄存器号

rd: 目的寄存器号

shamt: 移位的位数 (其它指令为00000)

funct: 功能码 (扩展操作码)

【移位指令:】

例如: `sll t2,s0,4`

rs: 没有使用

rt: 源寄存器 (\$s0)

rd: 目标寄存器 (\$t2)

shamt: 移位量 (4)

• I型

op	rs	rt	constant or address
6 bits	5 bits	5 bits	16 bits

立即数运算和load/store指令

rt: 源寄存器号 (store指令) 或目的寄存器号 (立即数运算和load指令)

立即数: -215 — +215 - 1

地址: rs中的基地址加上偏移量

• J型

op	address
6 bits	26 bits

跳转 (j和jal) 目标地址可以是代码段的任意位置

三、寄存器用法

a0–a3: 参数寄存器 (reg's 4 - 7)

v0,v1: 结果返回值寄存器(reg's 2 and 3)

t0–t9: 临时寄存器, 用于存储临时值, 可以由被调用者重写 (reg8 - 15)

s0–s7: 保留寄存器, 用于存储变量, 必须由被调用者保存/恢复 (reg16 - 23)

gp: 静态数据的全局指针(reg28)sp: 栈指针(reg 29)

fp: 帧指针(reg30)ra: 返回地址(reg 31)

MIPS 机器语言

名字	格式	举例						注释
add	R	0	18	19	17	0	32	add \$s1, \$s2, \$s3
sub	R	0	18	19	17	0	34	sub \$s1, \$s2, \$s3
addi	I	8	18	17	100			addi \$s1, \$s2, 100
lw	I	35	18	17	100			lw \$s1, 100(\$s2)
sw	I	43	18	17	100			sw \$s1, 100(\$s2)
字段宽度		6 位	5 位	5 位	5 位	5 位	6 位	所有 MIPS 指令均为 32 位
R 型	R	op	rs	rt	rd	shamt	funct	算术指令格式
I 型	I	op	rs	rt	address			数据传送指令格式

类别	指令	示例	含义	注释
数据传输	取字	lw \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将一个字从内存中取到寄存器中
	存字	sw \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	将一个字从寄存器中取到内存中
	取半字	lh \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将半个字从内存中取到寄存器中
	取无符号半字	lhu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将半个字从内存中取到寄存器中
	存半字	sh \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	将半个字从寄存器存到内存中
	取字节	lb \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将一个字节从内存取到寄存器中
	取无符号字节	lbu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	将一个字节从内存取到寄存器中
	存字节	sb \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	将一个字节从寄存器存到内存中
	取链接字	ll \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	取字作为原子交换的前半部
	存条件字	sc \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1;$ $\$s1 = 0 \text{ or } 1$	存字作为原子交换的后半部分
	取立即数的高位	lui \$s1,20	$\$s1 = 20 * 2^{16}$	取立即数并放到高16位
逻辑	与	and \$s1,\$s2,\$s3	$\$s1 = \$s2 \& \$s3$	三个寄存器操作数按位与
	或	or \$s1,\$s2,\$s3	$\$s1 = \$s2 \mid \$s3$	三个寄存器操作数按位或
	或非	nor \$s1,\$s2,\$s3	$\$s1 = \sim(\$s2 \mid \$s3)$	三个寄存器操作数按位或非
	立即数与	andi \$s1,\$s2,20	$\$s1 = \$s2 \& 20$	和常数按位与
	立即数或	ori \$s1,\$s2,20	$\$s1 = \$s2 \mid 20$	和常数按位或
	逻辑左移	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$	根据常数左移相应位
	逻辑右移	srl \$s1,\$s2,10	$\$s1 = \$s2 \gg 10$	根据常数右移相应位
条件分支	相等时跳转	beq \$s1,\$s2,25	if($\$s1 == \$s2$) go to PC + 4 + 100	相等检测; 和 PC 相关的跳转
	不相等时跳转	bne \$s1,\$s2,25	if($\$s1 \neq \$s2$) go to PC + 4 + 100	不相等检测; 和 PC 相关的跳转
	小于时置位	slt \$s1,\$s2,\$s3	if($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	比较是否小于; beq, bne
	无符号数比较小于时置位	sltu \$s1,\$s2,\$s3	if($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	比较是否小于无符号数
	无符号数比较小于立即数时置位	slti \$s1,\$s2,20	if($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$	比较是否小于常数
	无符号数比较小于无符号立即数时置位	sltiu \$s1,\$s2,20	if($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$	比较是否小于无符号常数
无条件跳转	跳转	j 2500	go to 10000	跳转到目标地址
	跳转至寄存器所指位置	jr \$ra	go to \$ra	用于 switch 语句, 以及过程调用
	跳转并链接	jal 2500	$\$ra = \text{PC} + 4$; go to 10000	用于过程调用

计算机最基本功能

一、寻址方式

1. 立即数寻址

对于I型指令：

汇编语言： `addi s1,s2, 10`

表示： $s1 = s2 + E(10)$

需要含有立即数，并且操作数是指令自身的16位字段

2. 寄存器寻址

对于R型指令：

汇编语言： `add t0,s1, s2`

表示： $t0 = s1 + s2$

直接对寄存器进行操作，操作数是寄存器。

3. 基址寻址

对于I型指令：

Rs 中存放基址寄存器，因为立即数只有16位，所以我们将其带符号拓展为32位，然后与基址相加，就可以得到目标数的地址。

常见的基址寻址指令有：lw, sw, lh, sh, lb, lbu等

4. PC相对寻址

对于I型指令：

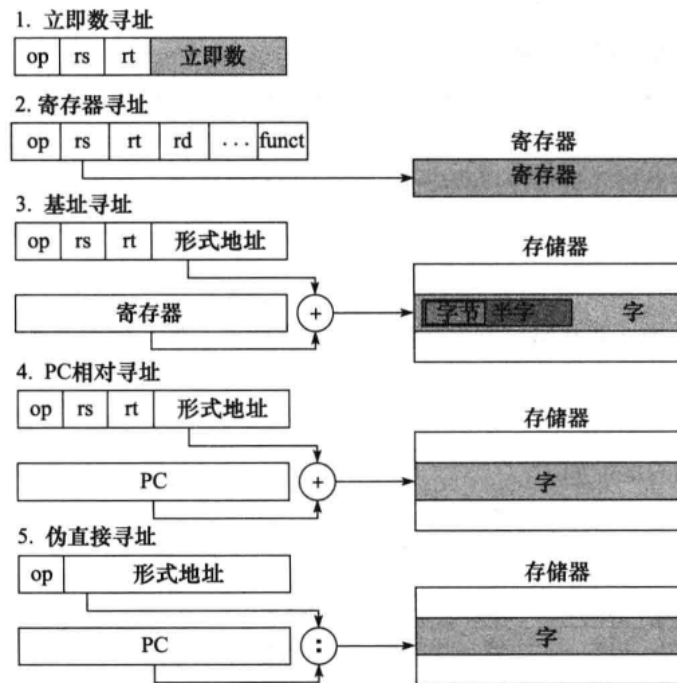
将16位的立即数带符号拓展为32位后，将其左移两位（此时它的最低两位就是0），然后与PC寄存器地址相加，所得为目标值的地址

常见相对寻址指令有：beq, bne

5. 伪直接寻址

对于J型指令：

对于26位的立即数直接左移两位，此时它为28位，而且它的最低两位为0，这时我们取PC的高四位地址做为目标地址的高四位，其余28位为得到的立即数，这个地址为目标值的地址。



二、支持程序的基本运行功能

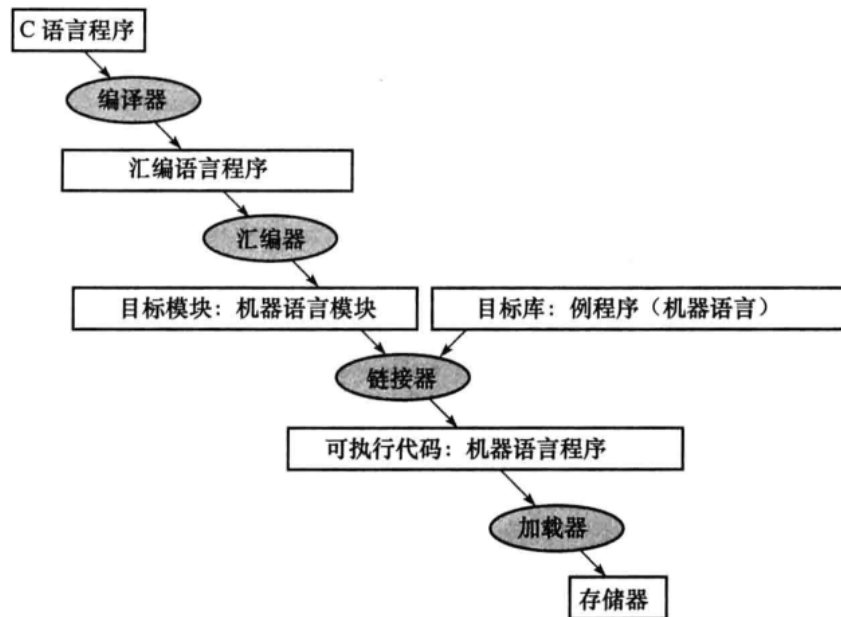
基本块：

没有分支（可能出现在末尾者除外）并且没有分支目标/分支标签（可能出现在开始者除外）的指令序列。

子函数调用过程：

- 1) 将参数放在过程可以访问的位置。
- 2) 将控制转交给过程。
- 3) 获得过程所需的存储资源。
- 4) 执行需要的任务。
- 5) 将结果的值放在调用程序可以访问的位置。
- 6) 将控制返回初始点，因为一个过程可能由一个程序中的多个点调用。

三、从程序到代码



支持基本的数据

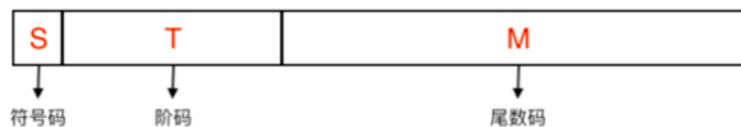
一、数的表示

原码、反码、补码

各种进制

二、浮点数

IEEE754标准



类 型	符号符	阶码	尾数码	总位数	偏置值	
					十六进制	十进制
短浮点数	1	8	23	32	7FH	127
长浮点数	1	11	52	64	3FFH	1023
临时短浮点数	1	15	64	80	3FFFH	16383

https://blog.csdn.net/weixin_47713503

IEEE 754 浮点数标准中的阶码

阶码是用移码表示的，移码的定义：移码 = 真值 + 偏置值；

1. 普通情况下，移码的偏置值为 $2^{(n-1)}$ ，8 位的移码的偏置值为 $2^{(8-1)} = 128D = 1000\ 0000B$ ；

例如： $-127D = -0111\ 1111B$ ，其移码为 $-0111\ 1111 + 1000\ 0000 = 0000\ 0001$ ；

2. 在 IEEE 754 标准中，移码的偏置值是 $2^{(n-1)}-1$ ，8 位的移码的偏置值为 $2^{(8-1)}-1 = 127D = 0111\ 1111B$ ；

例如: $-126D = -0111\ 1110B$, 其移码为 $-0111\ 1110 + 0111\ 1111 = 0000\ 0001$

注意:

8 位中, $-127D$ 和 $128D$ 这 2 个阶码有特殊用途, 一般不在正常讨论范围 ($-126 \sim 127$), 他们的移码分别是:

$-127D: -0111\ 1111 + 0111\ 1111 = 0000\ 0000$;

$128D: 1000\ 0000 + 0111\ 1111 = 1111\ 1111$;

IEEE 754 浮点数标准中的尾数

尾数码部分采用原码表示, 且尾数码隐含了最高位 1, 在计算时我们需要加上最高位 1, 即 $1.M$ 。

例如: 有一个浮点数, 真值为 $0.11B$, 那么其简略版的 IEEE 754 标准表示为 (忽略符号码和阶码):

$S(1)\ E(8)\ 100\ 0000\ 0000\ 0000\ 0000\ 0000$

即当 $0.11B$ 这个数在记录为 IEEE 754 标准浮点数时, 会这样处理, 令 $0.11 = 1.1 * 2^{(-1)}$, 尾数码是 $1.1000...$, 然后隐含最高位 1, 即 $1.1000...$ 。

IEEE 754 标准浮点数的真值

Value (值) = sign (符号位) $\times$ exponent (阶码) 【基数为 2】 $\times$ fraction (分数值/小数)

特殊情况

一般我们正常探讨的阶码区间是 $-126D \sim 127D$, 而真值 $-127D$ 的阶码为 $0000\ 0000B$, 真值 $128D$ 的阶码为 $1111\ 1111B$ 。

- 阶码全为 0 (阶码真值为 -127)
 - 当阶码全为 0, 尾数不全为 0, 表示非规格化小数, 用来表示比最小绝对值还要小的数, 即尾数码隐含的最高位不是 1, 而是 0; 阶码真值固定为 -126 , 而非 -127 ;
 - 当阶码全为 0, 尾数全为 0, 表示真值 ± 0 ;
- 阶码全为 1 (阶码真值为 128)
 - 当阶码全为 1, 尾数全为 0, 表示正负无穷大 $\pm \infty$
 - 当阶码全为 1, 尾数不全为 0, 表示非数值 NaN (Not a Number) 如 $0/0$, $\infty - \infty$ 等非法运算的结果即为 NaN

支持基本的运算——加法器

- 串行加法器——>超前进位加法器

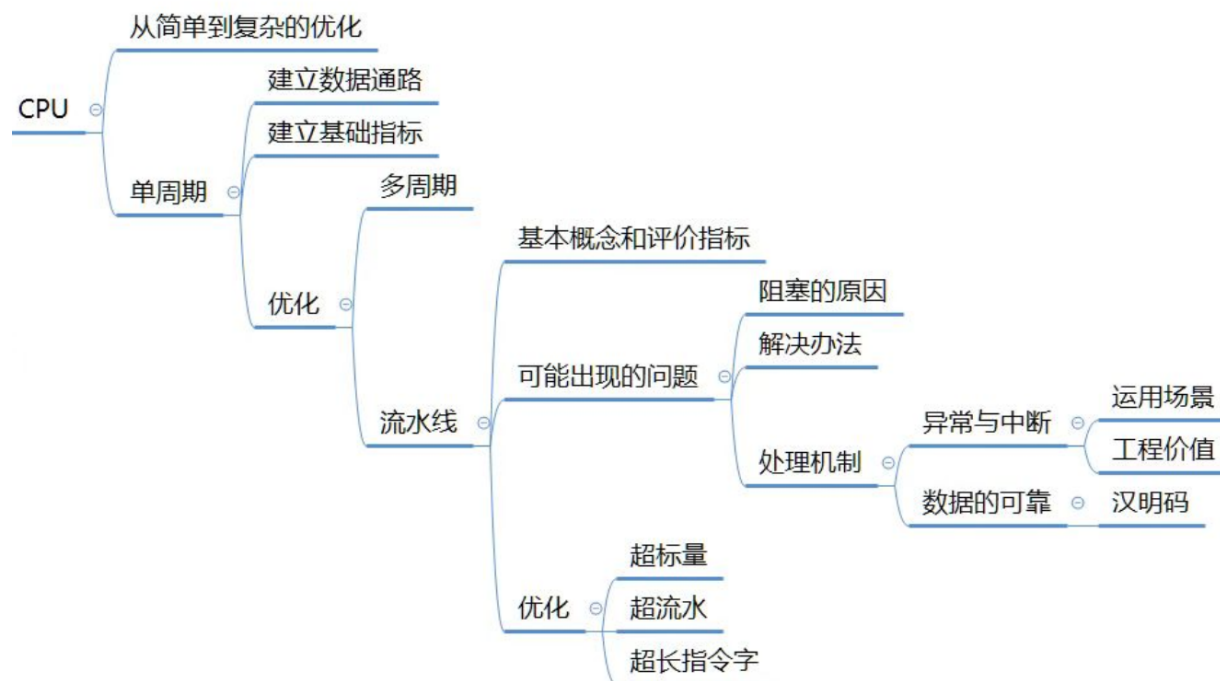
超前进位方式是快速的, 与位数无关。但随着加法器位数的增加, 逻辑表达式会变待越来越长, 这会使电路结构变得很复杂。因此, 当位数较多时采用全先行进位是不现实的。

更多位数的加法器可通过将超前进位加法器串接起来实现。例如, 对于 16 位加法器, 可以分成 4 组, 组内为 4 位先行进位, 组间串行进位。为了进一步提高运算速度, 也可以采用组内和组间都并行的进位方式。因为两级先行进位加法器组内和组间都采用先行进位方式, 其延迟和加法器的位数没有关系。所以, 通常采用两级或多级先行进位加法器。

- 加减法溢出判断——双符号位

若两个符号位相同，则无溢出；否则溢出。

计算机的构成之CPU



单周期CPU —— 一个时钟周期执行一条指令

一、数据通路：

- R型指令（加减法）—— `add t1,t2, $t3`
 1. 从**指令存储器**中取出指令，**PC**自增。
 2. 从**寄存器堆**中读出两个寄存器的值。同时，主控制单元计算出各控制信号的状态。
 3. **ALU**根据funct字段确定ALU功能，对从寄存器堆读出的数据进行操作。
 4. 将ALU的结果写回**寄存器堆**，根据指令的 [15: 11] 选择目标寄存器。
- 取数指令 —— `lw t1, offset(t2)`
 1. 从**指令存储器**中取出指令，**PC**自增。
 2. 从**寄存器堆**中读出寄存器 `$t2` 的值。
 3. **ALU**从寄存器堆读出的值与符号扩展后的指令低16位相加。
 4. 将ALU的结果作为**数据存储器的地址**。
 5. 存储单元的数据写回**寄存器堆**，根据指令的 [20: 16] 选择目标寄存器。
- beq指令 —— `beq t1t2 offset`
 1. 从**指令存储器**中取出指令，**PC**自增。
 2. 从**寄存器堆**中读出寄存器 `t1`、`t2` 的值。
 3. **ALU**将寄存器堆读出的两个数相减。`PC+4`的值与符号扩展并左移2位后的指令低16位相加，结果即分支目标地址。

4. 根据ALU的零输出决定哪个加法器的结果存入PC中

二、控制信号：

输入或输出	信号名	R 型	lw	sw	beq
输出	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

控制多选器 MUX：

RegDst：写寄存器的目标寄存器号来自rd

ALUSrc：第二个ALU操作数是指令低16位的符号扩展

MemtoReg：写入存储器的数据来自于数据存储器

控制寄存器堆和存储器读写的信号：

RegWrite：寄存器堆写使能有效

MemRead：数据存储器读使能有效

MemWrite：将写入数据输入端的数据写入到用地址指定存储器单元中取

决定是否转移：

Branch：决定是否分支

ALU的两位控制信号：

ALUOp0、ALUOp1：运算类型

三、单周期CPU的缺点

时钟周期对所有指令等长，要由执行时间最长的指令（lw指令决定）。lw指令依次使用了5个功能单元——指令存储器、寄存器堆、ALU、数据存储器、寄存器堆。

流水线CPU —— 指令重叠

一、基本概念

流水线是一种实现多条指令重叠执行的技术。它所带来的性能提高是通过增加指令的吞吐率，而不是减少单条指令的执行时间实现的。

五级流水线被分为五个阶段：

取指（IF）：从存储器取指令，更新PC

译码 (ID)：指令译码，从寄存器堆读出寄存器的值

执行 (EX)：运算指令——进行算数逻辑运算；访存指令——计算存储器的地址

访存 (MEM)：load指令——从存储器读数据；store指令——将数据写入存储器

回写 (WB)：将数据写入寄存器堆

所有流水级都只花费一个时钟周期的时间，因此，时钟周期必须能够满足最慢操作的执行需要。

二、问题与解决

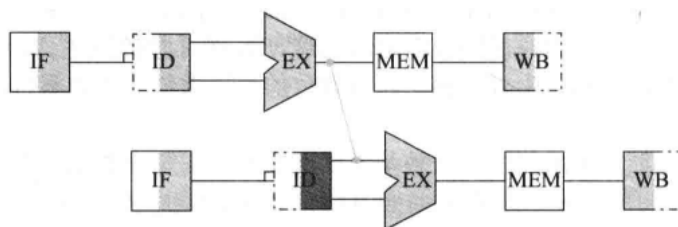
(一) 流水线冒险

1. 结构冒险：硬件不支持多条指令在同一时钟周期执行
2. 数据冒险：因无法提供指令执行所需数据而导致指令不能在预定的时钟周期内执行
3. 控制冒险：（分支冒险）因为取到的指令不是所需要的，而导致指令不能在预定的时钟周期内执行。

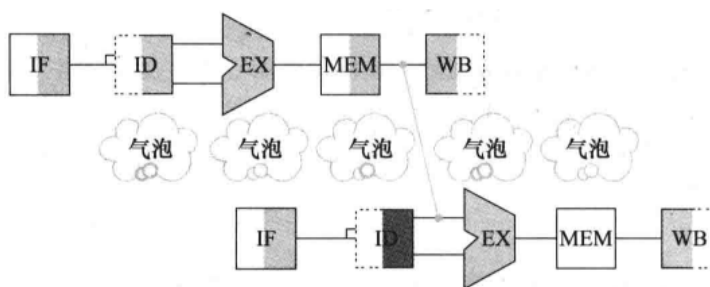
(二) 处理机制

数据冒险：旁路与阻塞

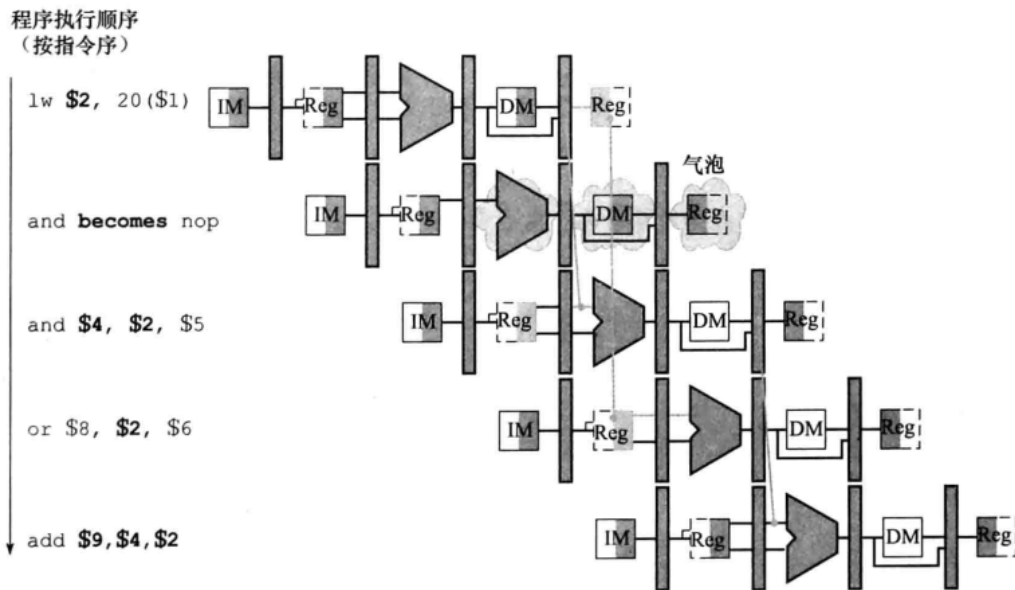
1. 在EX-EX之间建立旁路：



2. 在MEM-EX之间建立旁路：



3. load—use：当一条指令试图读取一个由前一条装载指令（lw）读入的寄存器时，就无法使用旁路解决冒险了。此种情况的解决方法是：在两条指令之间插入空指令，如下图。在第四个时钟周期中，通过将and指令变成nop插入了一个气泡。（注：and的IF和ID在第2、3周期，但EX被推迟到第5个周期。or的IF级仍在第3个周期，但ID被推迟到第5个周期）在插入气泡后，所有的相关性沿时间前进，冒险不再发生。



控制冒险

1. 假定分支不发生：

遇到分支指令时，假定分支不发生，并继续执行顺序的指令流。如果分支发生，就丢弃已经读取并译码的指令，并按分支目标继续执行。

2. 缩短分支的延迟：

为了将分支决策提前，需要提前两个动作：计算分支目标地址和判断分支条件

◦ 计算分支目标地址：

将分支目标地址的计算从EX级移动到ID级

◦ 判断分支条件：

在ID级指令译码后，决定是否需要将所需数据旁路到相等检测单元进行相等检测。

ID级进行分支比较所需要的数据可能在后面才产生，可能发生数据冒险，需要阻塞流水线。

无条件直接转移——j：无停顿

无条件间接转移——jr：停顿一个周期，ID阶段即可获得目标地址

条件转移——beq：停顿一个周期，在ID阶段增加额外的比较电路

3. 动态分支预测：

根据运行信息在运行中进行分支预测。

分支预测缓存（分支历史记录表）：一小块按照分支指令的低位地址索引的存储器区，其中包含一位或多位数据用以说明最近是否发生过分支。

4. 分支延迟时间槽：紧跟延迟分支指令的时间片。用不影响分支的一条指令填充到该时间片中。

三、异常与中断

- 异常：控制流中任何意外的改变，而无论其产生原因来自处理器内部还是外部
- 中断：由外部引起的事件

MIPS异常处理

EPC：32位寄存器，用于保存发生异常的指令地址（都需要）

1. Cause：记录异常原因的寄存器。

所有异常使用同一入口地址，操作系统根据状态寄存器确定异常原因。

2. 向量中断：由异常原因决定中断控制转移地址的中断

操作系统通过异常向量地址得知异常原因。在向量中断中，控制器被转移到由异常原因决定的地址处。

异常处理的流程主要包括以下步骤：

1. 保护现场，将各个寄存器的值压栈，以便处理完之后回到原来的指令流。
2. 根据硬件设置的寄存器标志，判断是什么异常并执行具体的异常处理函数。
3. 恢复现场，将栈里保存的寄存器的值再写回。
4. 跳转到正常指令流断点，回到CPU正常的指令流。

流水线中的异常处理实现

当异常在EX级被检测出来，可以用EX.Flush信号避免EX级的指令在WB级写回结果。

异常处理的最后一步是将导致异常的指令地址保存在EPC中（实际上保存的是原始地址+4，因此异常处理历程必须先从前面的地址中减去4）

一个时钟周期内可能发生多个异常，通常的解决办法是对异常划分优先级。EPC捕捉中断指令的地址，MIPS的cause寄存器在一个时钟周期内记录下所有可能的异常。如果在cause寄存器中保存多个异常，当优先级最高的异常处理之后，会继续导致硬件中断，从而处理后面的异常。

01 硬件/软件接口 硬件与操作系统必须协同工作才能按照我们期望的方式处理异常。硬件一般暂停指令流中导致异常的指令，同时执行完该指令前的所有指令，清除该指令后的所有指令，并且设置一个寄存器描述异常发生的原因，保存导致异常发生的指令的地址，然后跳转到预先确定的地址开始执行。操作系统则查看异常发生的原因并采取相应的操作。对于一个未定义指令异常、硬件错误异常或算术溢出异常，操作系统通常终止执行的程序并返回原因描述。对于I/O设备请求或操作系统服务调用，操作系统保存程序的当前状态，执行期望的任务，然后重新载入程序继续运行。在I/O设备请求的情况下，我们可能需要在继续执行发出I/O设备请求的任务前先运行另一个任务，因为该任务一般在I/O完成之后才能继续执行。这就是保存和恢复任务状态如此重要的原因。一个最重要且频繁出现的异常是页缺失与TLB异常。第5章描述了更多关于这些异常及其处理的细节。

四、提高可靠性——冗余技术

纠正一位错、检测两位错的汉明编码

步骤：

1. 对数据部分从左到右由1开始依次编号（这跟通常采用的从最右边开始由0开始编号的做法相反）
2. 将所有编号为2的整数次幂的位标记为奇偶校验位（1, 2, 4, 8, 16,）
3. 其他剩余位置用作数据位（3, 5, 6, 7, 9, 10, 11,）
4. 奇偶校验位的位置决定了其对应的数据位：

- 校验位1 (0001) : 对应数位编号的二进制形式最右边一位均为1 (检查第1, 3, 5, 7, 9, 11,)
- 校验位2 (0010) : 对应数位编号的二进制形式最右边第二位均为1 (检查第2, 3, 6, 7, 10, 11, 14, 15,)
- 校验位4 (0100) : 对应数位编号的二进制形式最右边第三位均为1 (检查第4, 5, 6, 7, 12, 13, 14, 15,)

5. 设置奇偶校验位, 对各组进行偶检验

五、流水线优化

超标量:

超级标量是指cpu内一般能有多条流水线, 这些流水线能够并行处理。

在单流水线结构中, 指令虽然能够重叠执行, 但仍然是顺序的, 每个周期只能发射(issue)或退休(retire)一条指令。超级标量结构的cpu支持指令级并行, 每个周期可以发射多条指令(2-4条居多).这样,可以使得cpu的IPC(InstructionPer Clock) > 1, 从而提高cpu处理速度。超级标量机能同时对若干条指令进行译码, 将可以并行执行的指令送往不同的执行部件, 在程序运行期间, 由硬件(通常是状态记录部件和调度部件)来完成指令调度。超级标量机主要是借助硬件资源重复(例如有两套译码器和ALU等)来实现空间的并行操作。

超流水:

超级流水线是将cpu处理指令的操作进一步细分, 增加流水线级数来提高频率。

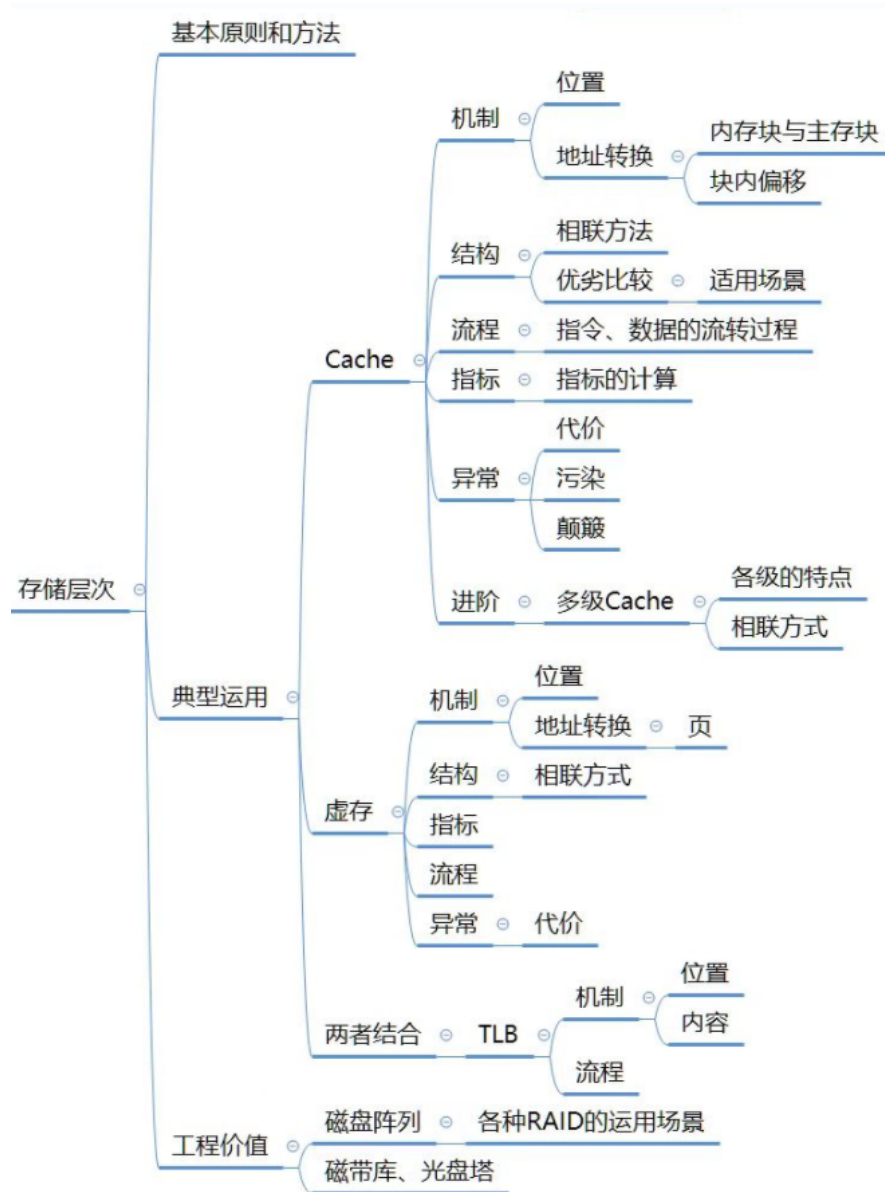
超级流水线又叫做深度流水线, 它是提高cpu速度通常采取的一种技术。CPU处理指令是通过Clock来驱动的, 每个clock完成一级流水线操作。每个周期所做的操作越少, 那么需要的时间久越短, 时间越短, 频率就可以提得越高。所以超级流水线就是将cpu处理指令是得操作进一步细分, 增加流水线级数来提高频率。频率高了, 当流水线开足马力运行时平均每个周期完成一条指令 (单发射情况下), 这样cpu处理得速度久提高了。当然, 这是理想情况下, 一般是流水线级数越多, 重叠执行的执行就越多, 那么发生竞争冲突得可能性就越大, 对流水线性能有一定影响。

超长指令字:

一类可以同时启动多个操作的指令集, 其中操作在单个指令中相互独立, 并且一般都有独立的操作码字段。

超长指令字是一条指令来实现多个操作的并行执行, 之所以放到一条指令是为了减少内存访问。通常一条指令多达上百位, 有若干操作数, 每条指令可以做不同的几种运算。哪些指令可以并行执行是由编译器来选择的。通常VLIW机只有一个控制器, 每个周期启动一条长指令, 长指令被分为几个字段, 每个字段控制相应的部件。由于编译器需要考虑数据相关性, 避免冲突, 并且尽可能利用并行, 完成指令调度, 所以硬件结构较简单。

计算机的构成之存储层次



基本原则和方法

时间局部性：某个数据项在被访问之后可能很快再被访问的特性。

空间局部性：某个数据项在被访问之后，与其地址相近的数据项可能很快被访问的特性。

存储器层次结构：

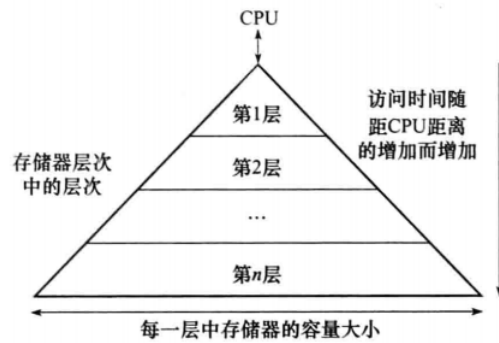


图 5-3 这幅图说明了存储器层次结构：离处理器越远，容量越大。当采用合适的操作机制时，这种结构允许处理器的访问时间主要由层次结构中的第 1 层来决定，而整个存储器的容量则和第 n 层一样大。本章的主题就是要实现这种结构。尽管本地磁盘一般位于存储器层次结构的底层，但是一些系统会使用磁带或者局域网内的文件系统作为层次结构的更下一层

(除非数据在第 $i+1$ 层存在，否则绝不可能在第 i 层存在。)

存储器技术

- SRAM：cache。非破坏性读出。易失性存储器。
- DRAM：主存储器。破坏性读出，每隔一段时间需要刷新。易失性存储器。
- 闪存：电可擦除的可编程只读存储器（EEPROM）。
- 磁盘存储器：
 - 磁道：磁盘表面的数万个同心圆环中的任意一个圆环
 - 扇区：构成磁盘上磁道的基本单位，是磁盘上数据读写的最小单位
 - 平均存取时间=寻道时间（磁头移到目标磁道）+旋转延迟时间（磁头定位到读写扇区的时间）+传输时间（传输数据所花费的时间）

cache

一、机制和结构

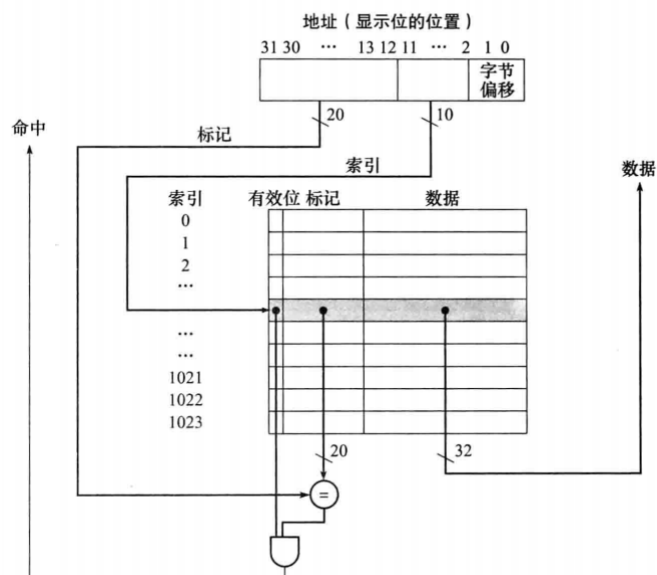
- 直接映射：

每个存储器地址仅仅对应到cache中的一个位置。

映射方法：（块地址）mod（cache中的块数）

地址可以划分为：

字节偏移（由块大小决定）、标记域（用来与cache中标记域的值进行比较）、cache索引（用来选择块）



直接映射的cache的总位数:

- 32 位地址。
- 直接映射 cache。
- cache 大小为 2^n 个块，因此 n 位被用来索引。
- 块大小为 2^m 个字 (2^{m+2} 字节)，因此 m 位用来查找块中的字，两位是字节偏移信息。

标记域的大小为

$$32 - (n + m + 2)$$

直接映射的 cache 总位数为

$$2^n \times (\text{块大小} + \text{标记域大小} + \text{有效位域大小})$$

由于块大小为 2^m 个字 (2^{m+5} 位)，同时我们需要 1 位有效位，因此这样一个 cache 的位数是

$$2^n \times (2^m \times 32 + (32 - n - m - 2) + 1) = 2^n \times (2^m \times 32 + 31 - n - m)$$

• 全相联:

块可以放置到cache中的任何位置

映射方法: 块可以放到任何位置

地址可以划分为:

字节偏移 (由块大小决定)、标记域 (用来与cache中标记域的值进行比较)

• 组相连:

块可以放置到cache中的部分位置

映射方法: (块地址) mod (cache中的组数)

地址可以划分为:

字节偏移 (由块大小决定)、标记域 (用来与cache中标记域的值进行比较)、cache组索引 (用来选择组)

不同结构的比较：

提高相连度的好处：能够降低缺失率；

缺点：增加了命中时间。直接映射cache只需要1个比较器，而当相连度越高，比较器就越多。组相联cache的开销包括额外的比较器以及由于对组里数据快进行比较和选择而产生的延迟。

适用场景：

在存储器层次结构中选择直接映射、组相联还是全相联映射取决于缺失代价和相联度实现代价的权衡情况，包括了时间和额外硬件开销。在片内拥有二级cache允许实现更高的相联度，这是因为命中时间不再关键，设计者也不用依靠标准SRAM芯片来构建块。除非容量很小，否则 cache不使用全相联映射方式，在小容量cache中，比较器的开销并不是压倒性的因素，其绝对缺失率的改进才是最明显的。

- **cache块的大小：**

使用较大的块可以降低缺失率，减少cache中与数据存储量相关的标记存储量，从而提高cache的效率。但是会带来缺失代价的增加。

二、cache流程

- 在cache中查找块

考虑在组相联（与直接映射是一个机理）的cache中查找一个cache块：组相联cache中每一块都包含一个地址标记用来给出块地址。在被选中的组中每一块的标记都要进行检测，从而判断是否和来自处理器的块地址相匹配。索引值用来选择包含所需地址的组，该组中所有块的标记都将被检索。

- cache块的替换

最近最少使用（LRU）：总是替换很长时间没有使用的块

- cache缺失的处理步骤：

(1) 指令缺失：把程序计数器（PC）的原始值（当前PC+4）送到存储器中。

数据缺失：处理器阻塞。

(2) 通知主存执行一次读操作，并等待主存访问完成。

(3) 写cache项，将从主存取回的数据写入cache中存放数据的部分，并将地址的高位（从ALU中得到）写入标记域，设置有效位。

(4) 重启指令执行第一步，重新取指，这次该指令/数据在cache中

- cache中的写

写直达：写操作总是同时更新cache和下一存储器层次，以保持二者的一致性。（写缓冲：为了提高性能，当一个数据在等待被写入主存时，先将它放入写缓冲中。写缓冲是一个等待写入主存数据的缓冲队列）。

写不分配：只更新主存中块的一部分，而不写入cache中。

写回：当发生写操作时，新值仅仅被写入cache块中，只有当修改过的块被替换时才需要写到较低层存储结构中。（也可以使用写缓冲）

写分配：数据块从主存中取回，并且在块中的恰当区域重写数据。

三、cache的性能

命中率、缺失率

$\text{CPU时间} = (\text{CPU执行时钟周期数} + \text{存储器阻塞的时钟周期数}) * \text{时钟周期}$

$\text{存储器阻塞的时钟周期数} = \text{读操作引起阻塞的时钟周期数} + \text{写操作引起阻塞的时钟周期数}$

$\text{读操作引起阻塞的时钟周期数} = (\text{读的次数/程序数}) * \text{读缺失率} * \text{读缺失代价}$

$\text{写操作引起阻塞的时钟周期数} = [(\text{写的次数/程序数}) * \text{写缺失率} * \text{写缺失代价}] + \text{写缓冲区阻塞}$

$\text{平均存储器访问时间 (AMAT)} = \text{命中时间} + \text{缺失率} * \text{缺失代价}$

四、cache的异常

cache缺失的处理在上一节介绍过。

污染：当cache块过大，将本不需要高速缓存的数据放到了cache中，则会导致该块污染。

颠簸：数据在cache和主存之间经常来回交换。

五、进阶：多级cache

使用一个大的二级cache来处理一级cache的缺失，从而降低了缺失代价。

一级cache致力于减少命中时间获得较短的时钟周期或者较少的流水级，二级cache则主要针对改善缺失率以减少长时间的访存代价。

虚拟存储器

一、机制与结构

虚拟存储器：

一种将主存用作辅助存储器高速缓存的技术。

功能特点：

1. 允许多个虚拟机共享同一个存储器。
2. 允许单用户程序使用超过主要存储器的容量。

相连方式采用全相联：

在虚拟存储器系统中，页表是一个独立的映射表，它用来索引存储器。除了表本身需要占用存储资源外，使用索引表还会引起额外的存储器访问。选择全相联映射和额外的页表有以下几个原因：

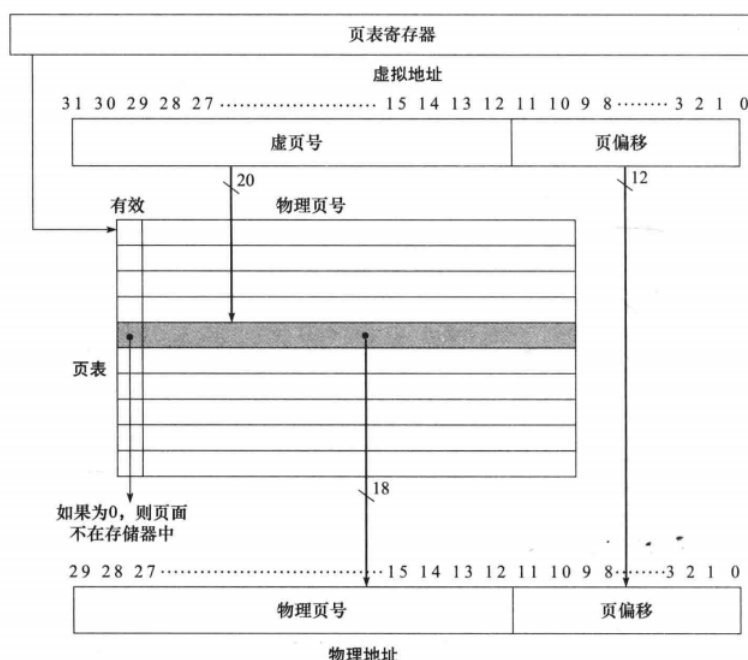
- 1) 全相联有其优越性，因为缺失的代价非常高。
- 2) 全相联允许软件使用复杂的替换策略以降低缺失率。
- 3) 全映射很容易被索引，而不需要额外的硬件，也不需要进行查找。因此，虚拟存储系统通常使用全相联映射。

地址转换：

(虚拟地址：虚拟空间的地址，当需要访问主存时，需要通过地址映射转换为物理地址。)

虚存地址：虚页号 页偏移

物理地址：物理页号 页偏移



二、流程

在存储器中，主存中的块（称为页）从一组地址（称为虚拟地址）映射到另一组地址（称为物理地址）。访问主存使用物理地址，而处理器产生虚拟地址。虚拟存储器和物理存储器都被划分成页，因此一个虚页被映射到一个物理页。当然，一个虚页也可能不在主存中，因此无法映射到物理地址；在这种情况下，页就被存在磁盘上。物理页可以被两个指向相同物理地址的虚拟地址共享。这种方法用来使两个不同的程序共享数据或代码。

• 页的存放

页表：保存虚拟地址和物理地址之间的转换关系。使用虚拟地址中的页号作索引，以找到相应的物理页号。

页表寄存器：指向页表首地址的寄存器。

用虚页号来索引页表以获得对应的物理地址部分。假定地址为32位。页表的首地址由页表指针给出。每一项的有效位指出了映射是否合法。如果该位为0，那么该页就不在主存中。

• 缺页

缺页：访问的页不在主存中。

当虚拟页的有效位关闭，会发生缺页故障。操作系统获得控制权。

一旦操作系统知道了引起缺页的虚拟地址，它必须完成以下三个步骤：

- 1) 使用虚拟地址查找页表项，并在磁盘上找到被访问的页的位置。
- 2) 选择替换一个物理页；如果被选中的页被修改过，需要在把新的虚拟页装入之前将这个物理页写回到磁盘上。

3) 启动读操作，将被访问的页从磁盘上取回到所选择的物理页的位置上。

注：缺页的代价很高，因此通常采用下面技术降低缺失率：

1) 增大页的容量以便利用空间局部性并降低缺失率。

2) 使用页表实现的虚拟地址和物理地址之间的映射采用全相联的方式，这样虚拟页就可以被放置到主存中的任何位置。

3) 操作系统使用类似LRU和访问位之类的技术来选择替换哪一页。

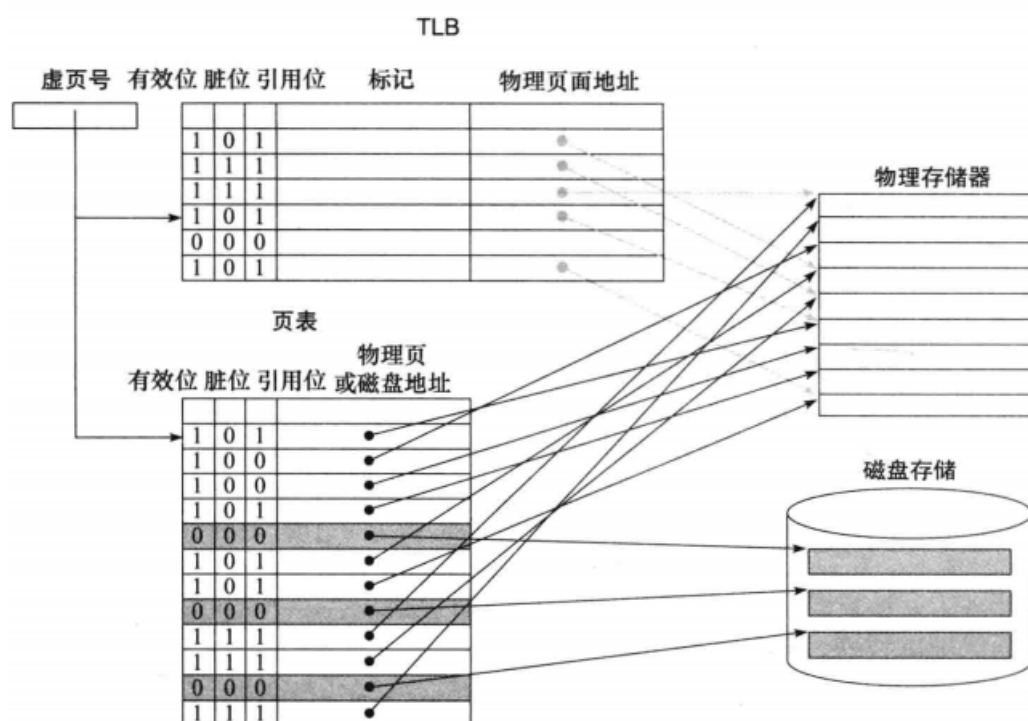
cache+虚存——TLB

一、机制

页表存在主存：每次访存至少需要两次——获得物理地址和获取数据。因此，采用cache思想设计TLB。

TLB：用于记录最近使用地址的映射信息的高速缓存，从而可以避免每次都要访问页表。

TLB组成：标记项（存放虚页号的一部分）、数据项（物理页号）、有效位（指明该页是否在主存中）、脏位（是否发生写操作）、引用位（当页被访问时置位，操作系统定期将引用位清零，用于跟踪哪些页被访问过以便替换）。



二、流程

每次访问，我们都要在TLB中查找虚页号。如果命中，物理页号就用来形成地址，相应的引用位被置位。如果处理器执行的是写操作，脏位同样要被置位。如果TLB发生缺失，我们必须判断是发生缺页还是仅仅是一次TLB缺失。如果该页在主存中，那么TLB缺失只是一次转换缺失。在这种情况下，处理器可以通过将页表中的变换装载到TLB中并且重新访问来进行缺失处理。如果该页不在主存中，TLB缺失就是一次真的缺页。在这种情况下，处理器调用操作系统的异常处理。由于TLB中的项比主存中的页数少得多，发生TLB缺失会比缺页频繁得多。

工程价值

一、磁盘阵列

RAID（独立冗余磁盘阵列）是指将多个独立的物理磁盘组成一个独立的逻辑盘，数据在多个物理盘上分割交叉存储、并行访问，具有更好的存储性能、可靠性和安全性。

RAID的分级如下所示。在RAID1 ~ RAID5几种方案中，无论何时磁盘损坏，都可随时拔出受损的磁盘再插入好的磁盘，而数据不会损坏，提升了系统的可靠性。

- RAID0：无冗余和无校验的磁盘阵列。
- RAID1：镜像磁盘阵列。
- RAID2：采用纠错的海明码的磁盘阵列。
- RAID3：位交叉奇偶校验的磁盘阵列。
- RAID4：块交叉奇偶校验的磁盘阵列。
- RAID5：无独立校验的奇偶校验磁盘阵列。

RAID0把连续多个数据块交替地存放在不同物理磁盘的扇区中，几个磁盘交叉并行读写，不仅扩大了存储容量，而且提高了磁盘数据存取速度，但RAID0没有容错能力。

为了提高可靠性，RAID1使两个磁盘同时进行读写，互为备份，若一个磁盘出现故障，可从另一磁盘中读出数据。两个磁盘当一个磁盘使用，意味着容量减少一半。

总之，RAID通过同时使用多个磁盘，提高了传输率；通过在多个磁盘上并行存取来大幅提高存储系统的数据吞吐量；通过镜像功能，提高安全可靠；通过数据校验，提供容错能力。

二、磁带库和光盘塔

磁带是传统的数据存储媒质，具有存储容量大的特点，但是随着对数据管理的即时性要求和数据安全性的要求，磁带已经不能再适应海量信息数据的存储和管理。

磁带对保存环境的要求高，磁带访问速度慢。

光盘库记录密度高、存储容量大，有海量存储器之称。